

Эссе по первым шести занятиям курса OTUS «Разработчик C++»

Занятие 1. Вводное занятие

Темы: организация курса; система контроля версий Git; работа с репозиторием на GitHub; ветки (branch) и коммиты; сравнение версий кода (diff). **Понятия:** система контроля версий хранит историю изменений кода, позволяет узнать, кто и когда что изменил, сравнить версии и объединить (merge) работу нескольких участников; репозиторий — хранилище проекта с историей изменений; ветка — независимая линия разработки, не затрагивающая основной код до момента слияния. **Результаты:** создал аккаунт и репозиторий на GitHub, завёл в нём ветки `test1` и `test2`, внёс в них разные изменения и сравнил через diff — увидел, как система контроля версий показывает построчные отличия. Итог задания — не код, а сам настроенный репозиторий, в котором я веду все последующие домашние задания.

Затруднение: не сразу разобрался с разницей между локальным репозиторием на компьютере и удалённым на GitHub — пришлось отдельно понять, как `git push`/`git pull` синхронизируют их между собой.

Занятие 2. Пишем свой Hello, World

Темы: первая программа на C++; сборка через CMake; разделение кода на заголовочный файл (объявление) и файл реализации. **Понятия:** заголовочный файл содержит только объявление функции — имя, параметры, тип возврата, — а файл реализации хранит её тело; это позволяет другим файлам подключать функцию через `#include`, зная только интерфейс. CMake — система автоматизации сборки, генерирующая нужные файлы по текстовому описанию `CMakeLists.txt`. **Результаты и устройство программы:** собрал исходную `hello_world.cpp` из материалов, затем по заданию вынес реализацию функции `todo_move_me_out()` в отдельную пару `move_me_out.h/.cpp`, не меняя код `main`, подключил заголовок через `#include` и добавил оба файла в `CMakeLists.txt`. Программа осталась рабочей — печатает приветствие и вызывает две вспомогательные функции, одна из которых (`example_function`) взята из материалов без изменений, а вторая вынесена мной.

Затруднение: сначала было неясно, зачем вообще разделять объявление и реализацию, если можно писать всё в одном файле. Стало понятнее на следующем занятии — именно на этом разделении строится идея библиотек.

Занятие 3. Базовый синтаксис языка C++

Темы: ввод-вывод `cin/cout`; пространства имён; диапазоны целочисленных и вещественных типов; условные операторы `if/switch`; циклы `while`, `do...while`, `for`; файловый ввод-вывод; приведение типов. **Понятия:** пространство имён группирует имена и избегает конфликтов (вся стандартная библиотека C++ лежит в `std`); диапазон типа — множество значений, которые он может хранить, а выход за границы (переполнение) даёт непредсказуемый результат. **Результаты:** отдельного задания с новой программой не было — разобрал и запустил примеры занятия `01_cin_cout.cpp`, `02_type_ranges_int.cpp`, `06_if.cpp`,

08_while_do_loop.cpp, 10_file_io.cpp и другие), собрал каждый через CMake и проверил вывод. Пример с диапазонами типов экспериментально показывает эффект переполнения — это наглядно объяснило, почему выбор типа данных не формальность.

Затруднение: дольше всего разбирался в разнице между `while` и `do...while` — стало понятно только после того, как подставил заведомо ложное условие в обе версии и сравнил результат: `do...while` всегда выполняет тело хотя бы один раз.

Занятие 4. Такие разные функции

Темы: способы передачи параметров (по значению, по ссылке, по константной ссылке, по указателю); параметры `main (argc, argv)`; перегрузка функций; параметры по умолчанию; рекурсия и стек вызовов. **Понятия:** передача по значению копирует аргумент, изменения копии не затрагивают исходную переменную; передача по ссылке даёт прямой доступ к исходной переменной (нужна для «выходных» параметров, например обмена значений); константная ссылка не копирует объект, но и не позволяет его изменить. Перегрузка — несколько функций с одним именем, но разными параметрами; выбор нужной делает компилятор. Рекурсия — вызов функции самой себя; без условия остановки приводит к переполнению стека. **Результаты:** без нового задания — разобрал примеры: `2_gcd_cin.cpp` (передача по значению, алгоритм Евклида), `6_swap.cpp` (передача по ссылке), `7_minigrep.cpp` (константная ссылка для строки), `3_args.cpp` (аргументы командной строки), `8_module.cpp` (перегрузка), `10_hanoi.cpp` (рекурсия на Ханойских башнях) и `11_fib.cpp` — наглядный «плохой» пример: наивная рекурсия для чисел Фибоначчи повторно пересчитывает одни и те же значения и растёт по времени экспоненциально.

Затруднение: сложно было понять, зачем нужно столько способов передачи параметров. Разобрался, сопоставив каждый способ с конкретной задачей: значение — когда данные не должны меняться; ссылка — когда функция обязана их изменить; константная ссылка — когда объект дорог для копирования, но менять его нельзя.

Занятие 5. Модульность. Линковка

Темы: файловый ввод-вывод через `<fstream>`; этапы сборки — препроцессинг, компиляция, ассемблирование, линковка; единица трансляции; статические и динамические библиотеки. **Понятия:** препроцессинг обрабатывает `#include`/`#define` текстовой заменой; компилятор проверяет код и превращает его в ассемблер; ассемблер даёт машинный код без связей между файлами; линковщик по таблицам символов связывает объектные файлы в единый исполняемый файл. Единица трансляции — один `.cpp`-файл со всем, что подставили в него директивы `#include`. Статическая библиотека копируется («вшивается») в каждый использующий её исполняемый файл; динамическая хранится и загружается отдельно, её можно подключать во время выполнения. **Результаты и устройство программы:** разобрал примеры файлового ввода-вывода, перегрузки, статической и динамической библиотек, а также сборку из нескольких единиц трансляции. По заданию написал игру «Угадай число» с таблицей рекордов, выполнив все пять дополнительных пунктов: флаги `-max` и `-level` (диапазон и уровень сложности, не сочетаются друг с другом), флаг `-table` (показать таблицу без игры), хранение только минимального результата на игрока и автоматическую перезапись худшего результата лучшим. Код разделён на три модуля: `cli_options.h/.cpp` разбирает аргументы командной

строки, `high_scores.h/.cpp` хранит таблицу как `std::map` «игрок → лучший результат» (что сразу решает оба условия про минимум и перезапись), а `main.cpp` связывает игру воедино, используя подходы из заготовок `check_value.cpp`, `random_value.cpp` и `argument.cpp`.

Затруднение: при тестировании программа зависала, если ввод заканчивался раньше угадывания – пришлось добавить проверку `std::cin.fail()` и корректно завершать программу при обрыве ввода.

Занятие 6. Подробнее о CMake

Темы: проблема переносимости сборки между окружениями; CMake как кроссплатформенная система сборки; понятие генератора; структура `CMakeLists.txt` от простого проекта до нескольких вложенных библиотек. **Понятия:** проблема «у меня работает» возникает из-за разных версий компилятора, библиотек, ОС и архитектур; закоммитить готовый проект одной IDE не решает проблему, так как жёстко привязывает сборку к одной платформе. CMake один раз описывает сборку в терминах, не зависящих от инструмента, а генератор создаёт из этого описания нужные сборочные файлы под конкретную платформу. **Результаты:** разобрал примеры от простейшего (`example1`, один исполняемый файл) до более сложных (`example4-example8`), которые показывают вынесение кода в отдельную библиотеку, генерацию заголовка с версией проекта из `config.h.in` и подключение внешней библиотеки через собственный `find`-модуль. Это напрямую опирается на понятия занятия 5 – единицу трансляции и библиотеки, – только описанные декларативно, а не собираемые вручную.

Затруднение: не сразу понял смысл файла `config.h.in` – казалось, проще написать обычный заголовок. Разобрался, когда увидел, что через этот механизм в код подставляются значения из `CMakeLists.txt` (например, версия проекта) без ручного редактирования кода.